Project Specification
SNU 4190.210, Programming Principles Fall 2025
Chung-Kil Hur
Due Date: 2025.12.25 (Thursday) 23:59

This file provides the semantics of the toy language for which you will implement an interpreter. Refer to `src/lib.rs` for a basic overview and important notes about the project.

Problem 1 Implement the `eval` function to serve as an interpreter for the programming language E described below in Rust.

$$\texttt{eval} : E \to V$$

FD	::=	$(x) \Rightarrow E$	(anonymous) function definition
B	::=	$(f = FD)$	function binding
E	::=	n	integer
	|	$\texttt{(}E\ \texttt{+}\ E\texttt{)}$	addition
	|	$\texttt{(}E\ \texttt{-}\ E\texttt{)}$	subtraction
	|	$\texttt{(}E\ \texttt{*}\ E\texttt{)}$	multiplication
	|	$\texttt{(}E\ \texttt{/}\ E\texttt{)}$	division
	|	$\texttt{(if0}\ E\ E\ E\texttt{)}$	conditional
	|	x	name
	|	$\texttt{(}E\ \texttt{(}E\texttt{))}$	function call
	|	$\texttt{(let}\ x := E\ \texttt{in}\ E\texttt{)}$	name binding of `val`
	|	$\texttt{(letfun}\ B^*\ \texttt{in}\ E\texttt{)}$	name binding of `def`
	|	$\texttt{(letfun (}\star = FD\texttt{) in }\star\texttt{)}$	anonymous function

- For ill-typed inputs, use the `panic!` macro to raise errors (e.g., `panic!("error message")`). Note that the error message itself will not be graded.
- X^* indicates that X can appear zero or more times.
- Addition, subtraction, multiplication, and division are performed using operations for Rust's `i64`. There is no need to explicitly handle division by zero, as Rust's `i64` division already raises an error in such cases.
- The `if0` E_1 E_2 E_3 expression first evaluates E_1 to a value v. If v equals 0, the result of E_2 is returned; otherwise, the result of E_3 is returned.

- Identifiers (names) `x` must be alphanumeric and cannot start with a digit.
- `let/letfun` clauses create a new scope, similar to a block in Scala. Name bindings `def` and `val` work in a similar manner as in Scala.
  - `(let` x `:=` E_1 `in` E_2`)` assigns the name x to the value obtained by evaluating E_1, then evaluates E_2.
  - `(letfun` B^* `in` E`)` defines functions in B^*, supporting mutual recursion within the same scope, and then evaluates E.
  - `(f = (x)` $\Rightarrow$ E) assigns the name `f` to the expression `E` with argument x. For example, `letfun (f x := (x - 1) in f(42))`.
  - Duplicate names cannot be defined within the esame scope.
- `Environment`(Env) is a collection of `Environment Entry`(EnvEntry). An `Environment Entry` is created whenever a new scope is introduced.
- For Problem 1, IO implementation is not required.
- Examples can be found in the test cases at the bottom of `src/eval.rs` and `src/eval_memo.rs`.
- `(letfun` $(\star = FD)^*$ `in` $\star$`)` is a syntactic sugar for anonymous functions using `letfun`.

Problem 2 (Extra Credit Problem) Add IO action to `interp` by implementing `defIO, runIO, readline` and `print` following:

E	::=	$\cdots$	
	|	(`runIO` E)	run an IO action
FD	::=	$\cdots$	
	|	(`funIO` x $\Rightarrow$ `(`IO^*`)` E)	define an IO action
IO	::=	(`readline` x)	read a line text and bind to x
	|	(`print` E)	print

- (f `= funIO (`x`)` $\Rightarrow$ `(`IO^*`)` E) defines a function that returns **IO actions**. f takes a parameter x like a normal function. Then, it executes IO^* sequentially, and finally E becomes the result of the IO action.
- The result of `(`f E`)` is **NOT** an evaluation of E. Instead, it returns an runnable *thunk* (pending action) which has a type of `IO[Value]` where the last E is `Value` type. From this stage, it does not actually runs (IO^*) inside f. Those will be run when `runIO` calls the thunk which will be explained below.

- `(runIO` E`)` actually **runs** an IO action when E returns IO action. (i.e. `IO[Value]`).
- `(readline` x`)` reads a line from the input (e.g. standard input) and binds it to x. The read value does not contain newline characters (`\n, \r, \r \n,` ...). You can assume that there will be only ASCII characters in the input.
- `(print` E`)` prints the result of E to the output (e.g. standard output). It does not automatically append newline characters.

You must use the `IOHandle` trait to abstract input and output. This allows us to test your code automatically without relying on actual stdin/stdout.

For example,

`(letfun ((funIO f1 = ((x) ⇒ (readline a)) (+ a x)) (funIO f2 = ((y) ⇒ (readline b)) (+ (runIO (f1 y)) b))) in runIO (f2 10))`

reads two numbers `a, b`, and evaluates to the value of `a + b + 10`.